



DESENVOLVIMENTO MOBILE

ALEX ZACHARIAS

ALEX@EINSTEINLIMEIRA.COM.BR

ARMAZENANDO DADOS - SQLITE

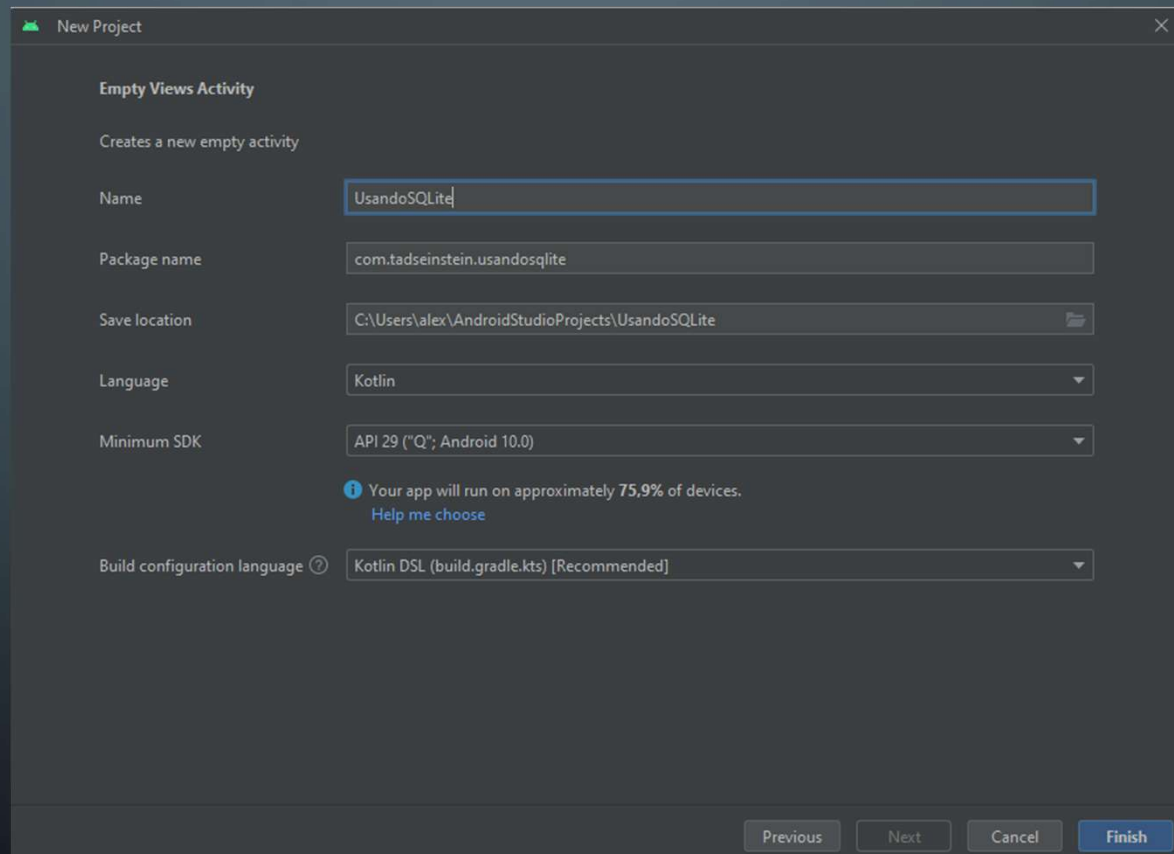
- O desenvolvimento para Android, seja em Java ou Kotlin, vem com uma implementação do SQLite embutida.
- O SQLite é um banco de dados SQL (de código aberto) bem leve, que armazena dados em forma de texto diretamente nos dispositivos.
- É feito para uso em casos onde os dados devem ficar armazenados localmente no próprio dispositivo, e não necessitam ser compartilhados (acessados por outros aparelhos).
- O SQLite não utiliza um servidor de banco de dados separado, todo seu controle fica dentro da própria aplicação.

ARMAZENANDO DADOS - SQLITE

- Para aprender sua utilização, vamos construir um aplicativo simples, onde cadastraremos pessoas informando apenas seu nome e idade.
- Faremos também uma lista para exibir os dados cadastrados.

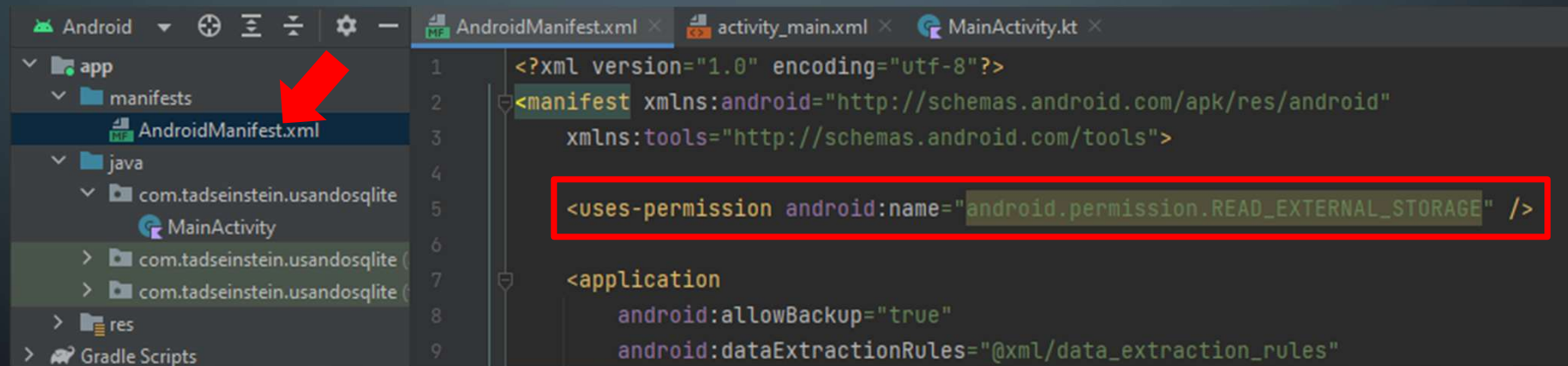
ARMAZENANDO DADOS - SQLITE

- Primeiro passo será criar um novo projeto, utilize **Empty Views Activity**:
Activity:



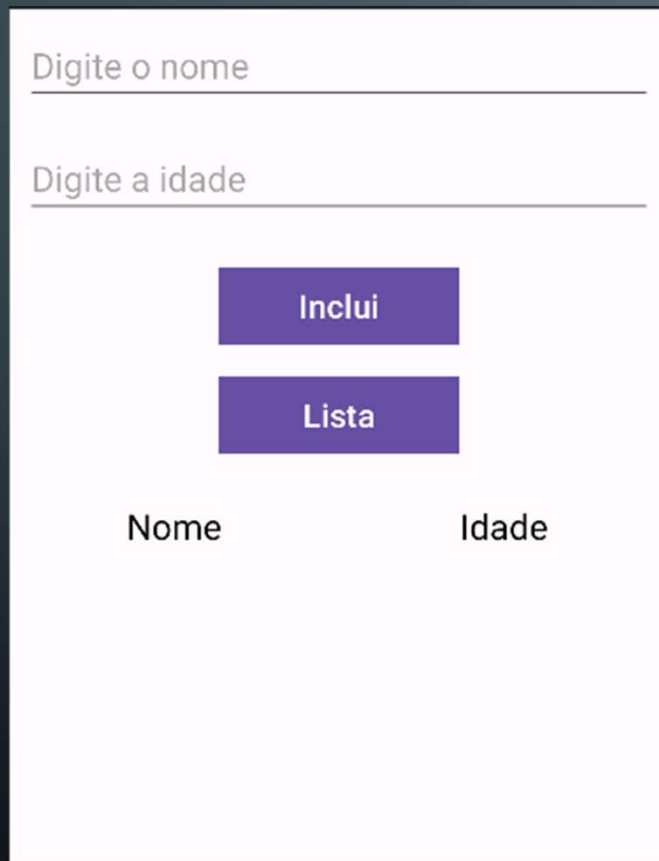
ARMAZENANDO DADOS - SQLITE

- Conceder permissão para que o aplicativo possa acessar o armazenamento de arquivos do Android. Abra o arquivo [AndroidManifest.xml](#) e adicione o código abaixo:



ARMAZENANDO DADOS - SQLITE

- Agora vamos montar a tela:



Digite o nome

Digite a idade

Inclui

Lista

Nome

Idade

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".MainActivity">

    <EditText
        android:id="@+id/edit_nome"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Digite o nome"
        android:textSize="22sp"
        android:layout_margin="10dp"/>

    <EditText
        android:id="@+id/edit_idade"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="number"
        android:layout_margin="10dp"
        android:textSize="22sp"
        android:hint="Digite a idade"/>
```

ARMAZENANDO DADOS - SQLITE

```
<Button
    android:id="@+id/btn_inclui"
    android:layout_width="150dp"
    android:layout_gravity="center"
    android:background="#2196F3"
    android:text="Inclui"
    android:textColor="#ffffff"
    android:textSize="20sp"
    android:layout_height="wrap_content"
    android:layout_marginTop="20dp"/>
```

```
<Button
    android:id="@+id/btn_lista"
    android:layout_width="150dp"
    android:layout_gravity="center"
    android:background="#2196F3"
    android:text="Lista"
    android:textColor="#ffffff"
    android:textSize="20sp"
    android:layout_height="wrap_content"
    android:layout_marginTop="20dp"/>
```

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content">

    <TextView
        android:id="@+id/txt_nome"
        android:textAlignment="center"
        android:layout_weight="1"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_margin="20dp"
        android:text="Nome\n\n"
        android:textSize="22sp"
        android:padding="10dp"
        android:textColor="#000000"/>

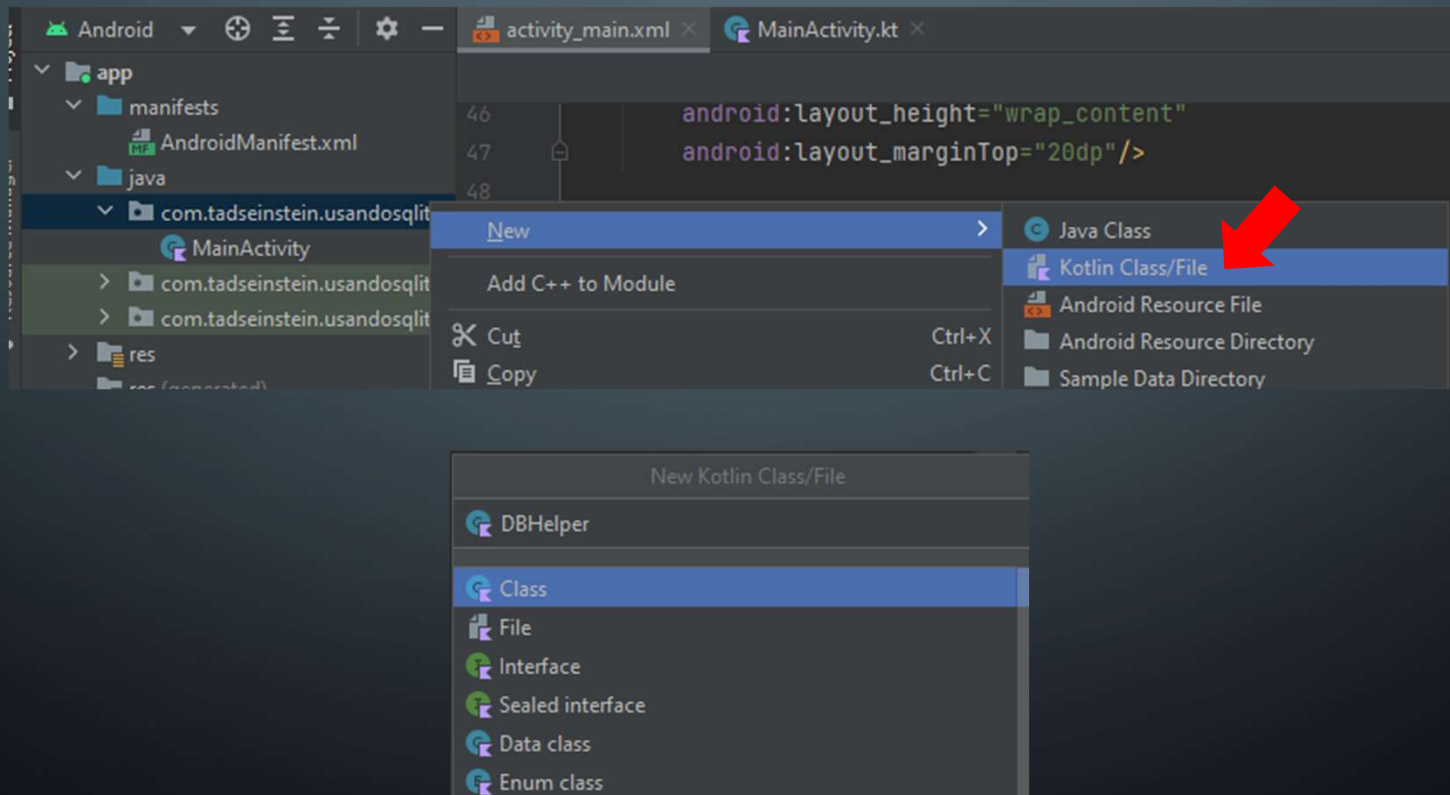
    <TextView
        android:id="@+id/txt_idade"
        android:textAlignment="center"
        android:layout_weight="1"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_margin="20dp"
        android:text="Idade\n\n"
        android:textSize="22sp"
        android:padding="10dp"
        android:textColor="#000000"/>

</LinearLayout>

</LinearLayout>
```

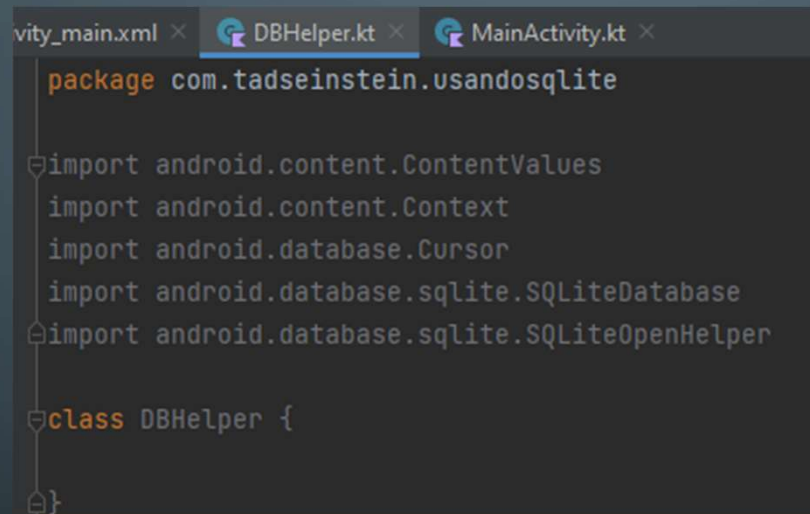
ARMAZENANDO DADOS - SQLITE

- Para controlar o acesso aos dados do SQLite vamos criar uma classe, chame ela de **DBHelper**:



ARMAZENANDO DADOS - SQLITE

- O Próximo passo é fazer o código da classe **DBHelper**.



```

vity_main.xml x  DBHelper.kt x  MainActivity.kt x
package com.tadseinstein.usandosqlite

import android.content.ContentValues
import android.content.Context
import android.database.Cursor
import android.database.sqlite.SQLiteDatabase
import android.database.sqlite.SQLiteOpenHelper

class DBHelper {
}

```

- Essas são as bibliotecas que precisamos importar.

ARMAZENANDO DADOS - SQLITE

```
class DBHelper(context: Context, factory: SQLiteDatabase.CursorFactory?) :  
    SQLiteOpenHelper(context, DATABASE_NAME, factory, DATABASE_VERSION) {  
  
    companion object{ // Aqui estamos definindo algumas variaveis  
  
        // Nome do banco de dados  
        private val DATABASE_NAME = "CADASTRO_DE_PESSOAS"  
  
        // Versão do banco de dados  
        private val DATABASE_VERSION = 1  
  
        // Nome da tabela  
        val TABLE_NAME = "pessoas"  
  
        // colunas da tabela  
        val ID_COL = "id"  
        val NAME_COL = "nome"  
        val AGE_COL = "idade"  
    }  
}
```

- Alteramos a classe **DBHelper** para que ela receba dois parâmetros e herde a classe **SQLiteOpenHelper**.

ARMAZENANDO DADOS - SQLITE

- Nossa classe **DBHelper** esta recebendo dois parâmetros:
 - O primeiro parâmetro é o contexto da aplicação, podemos utilizar **this**;
 - O segundo parâmetro é para criar um ponteiro customizado para acesso aos dados, como não estamos criando esse ponteiro, podemos passar **null**;
- Já a classe **SQLiteOpenHelper** recebe quatro parâmetros:
 - O Contexto que recebemos na classe **DBHelper**;
 - O nome do banco de dados;
 - O Cursor customizado quando necessário;
 - Um numero que indica a versão do banco de dados.

ARMAZENANDO DADOS - SQLITE

- Vamos criar um método para criação do banco:

```
// Método de criação do banco de dados  
// recebe qual banco deve ser criado  
override fun onCreate(db: SQLiteDatabase) {  
    // Carrega a query de criação da tabela em uma variável  
    val query = ("CREATE TABLE " + TABLE_NAME + " ("  
        + ID_COL + " INTEGER PRIMARY KEY, " +  
        NAME_COL + " TEXT," +  
        AGE_COL + " TEXT" + ")")  
  
    // método que executa a query  
    db.execSQL(query)  
}
```

ARMAZENANDO DADOS - SQLITE

- Como nossa classe `DBHelper` herda a classe `SQLiteOpenHelper`, tem alguns métodos que somos obrigados a declarar, independente de utilizarmos ou não. O método abaixo é um caso desse.

```
// Metodo que apaga a tabela se ja existir
override fun onUpgrade(db: SQLiteDatabase, p1: Int, p2: Int) {
    db.execSQL( sql: "DROP TABLE IF EXISTS " + TABLE_NAME)
    onCreate(db)
}
```

ARMAZENANDO DADOS - SQLITE

- Vamos adicionar registros no banco de dados:

```
// Método para adicionar registros no banco de dados
fun addName(name : String, age : String ){
    // Cria variável do tipo ContentValues
    val values = ContentValues()

    // Insere o nome e a idade na variável criada
    // utilizando a constante com o nome do campo
    // criada no início da classe
    values.put(NAME_COL, name)
    values.put(AGE_COL, age)

    // Cria a variável db como sendo
    // nosso banco de dados
    val db = this.writableDatabase

    // Insere o registro no banco de dados
    db.insert(TABLE_NAME, nullColumnHack: null, values)

    // Fecha o banco de dados
    db.close()
}
```

ARMAZENANDO DADOS - SQLITE

- Método que lê os dados da tabela e retorna um cursor com esses dados:

```
// Metodo para selecionar os dados da tabela
// ele retorna um cursos com os dados
fun getName(): Cursor? {

    // Cria a variável db como sendo o banco de dados
    // apenas com direitos de leitura dos dados
    val db = this.readableDatabase

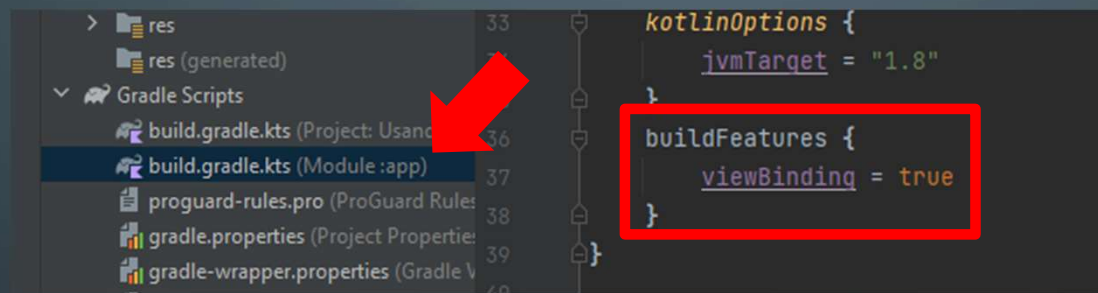
    // Retorna o cursor com os dados da tabela
    return db.rawQuery( sql: "SELECT * FROM " + TABLE_NAME, selectionArgs: null)

}
```

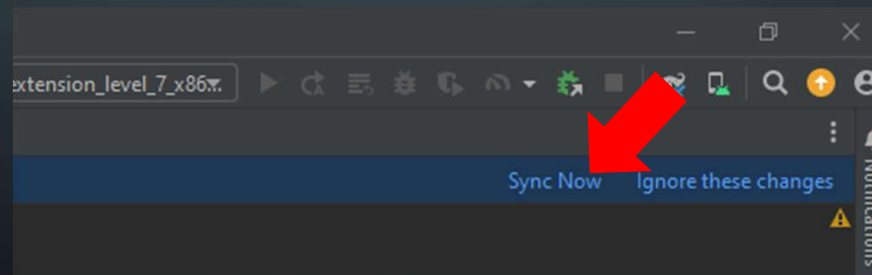
- Nossa classe para acesso aos dados esta pronta.

ARMAZENANDO DADOS - SQLITE

- Agora vamos alterar o código para acesso a parte visual.
- Primeiro precisamos fazer o **ViewBinding**:



- Após a alteração, clique em **Sync Now**:



ARMAZENANDO DADOS - SQLITE

- Faça o **binding** na **MainActivity**:

```
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.view.View
import com.tadseinstein.usandosqlite.databinding.ActivityMainBinding

class MainActivity : AppCompatActivity(), View.OnClickListener {
    private lateinit var binding: ActivityMainBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)

        binding.btnInclui.setOnClickListener(this)
        binding.btnLista.setOnClickListener(this)
    }

    override fun onClick(v: View) {

    }
}
```

ARMAZENANDO DADOS - SQLITE

- Agora vamos criar as funções que serão executadas ao clicar nos botões:

```
override fun onClick(v: View) {  
    if (v.id == R.id.btn_inclui) {  
        botaoInclui()  
    }  
    else if (v.id == R.id.btn_lista) {  
        botaoLista()  
    }  
}  
  
private fun botaoInclui(){  
  
}  
  
private fun botaoLista() {  
  
}
```

ARMAZENANDO DADOS - SQLITE

- Código da função que será executado ao clicar no botão inclui:

```
private fun botaoInclui() {  
    // Conecta o banco de dados na variavel db  
    val db = DBHelper( context: this, factory: null)  
  
    // carrega os dados  
    val name = binding.editNome.text.toString()  
    val age = binding.editIdade.text.toString()  
  
    // Inclui no banco de dados  
    db.addName( name, age)  
  
    Toast.makeText( context: this, text: name + " adicionado no banco!", Toast.LENGTH_LONG).show()  
  
    // Limpa os EditText  
    binding.editNome.text.clear()  
    binding.editIdade.text.clear()  
}
```

ARMAZENANDO DADOS - SQLITE

- Código do botão Listar:

```
@SuppressWarnings("Range")
private fun botaoLista() {
    // Conecta o banco de dados na variável db
    val db = DBHelper(context: this, factory: null)

    // Chamamos o método getName para pegar os registros
    // e associá-los ao cursor
    val cursor = db.getName()

    // Move o cursor para o primeiro registro
    cursor!!.moveToFirst()

    // Manda os registros para a tela
    do {
        binding.txtNome.append(cursor.getString(cursor.getColumnIndex(DBHelper.NAME_COL)) + "\n")
        binding.txtIdade.append(cursor.getString(cursor.getColumnIndex(DBHelper.AGE_COL)) + "\n")
    } while (cursor.moveToNext())

    // Fecha o cursor
    cursor.close()
}
```

ARMAZENANDO DADOS - SQLITE

```
@SuppressWarnings("Range")
```

- O Código acima serve apenas para que o programa não acuse erro, ele faz com que erro de range numérico seja desconsiderado.
- o método `getColumnIndex` aceita -1, mas `DBHelper.nomedacoluna` nunca retorna -1, por isso o Kotlin dá um erro na linha abaixo se não utilizarmos o código acima.

```
binding.txtNome.append(cursor.getString(cursor.getColumnIndex(DBHelper.NAME_COL)) + "\n")
```